
Comparative Evaluation of Extended DMD and Neural ODEs for Trajectory Modeling in Linearly Immersible and Chaotic Dynamical Systems

Kieran Vlahakis* Elizabeth Won* Anish Goel* Sidd Ojha*

California Institute of Technology

ACM 270

1 Introduction

We wish to present a performance-based exploration of two methods discussed in this course. In particular, we are interested in evaluating the performance of state of the art nonlinear methods on a linearly immersible system, such as the Lotka-Volterra predator-prey system, against Extended Dynamical Mode Decomposition (EDMD) and sensitivity methods for ODEs. We then extend our analysis to include non-linearly immersible and chaotic systems, such as the Lorenz attractor.

2 Background

Before introducing the numerical experiments that we perform to probe this research question, we briefly review some of the relevant concepts discussed throughout the course [1] upon which the methods explored here build on.

2.1 Extended Dynamic Mode Decomposition (EDMD) and Kernel Dynamic Mode Decomposition (KDMD)

We first begin with a quick explanation of Dynamic Mode Decomposition, or DMD methods. Given time series data

$$x_1, \dots, x_{m+1} \in \mathbb{R},$$

we form matrices

$$\mathbf{X} = \begin{bmatrix} | & & | \\ x_1 & \dots & x_m \\ | & & | \end{bmatrix}$$

and

$$\mathbf{Y} = \begin{bmatrix} | & & | \\ x_2 & \dots & x_{m+1} \\ | & & | \end{bmatrix}$$

where

$$\mathbf{X}, \mathbf{Y} \in \mathbb{R}^{n \times m}.$$

The Exact DMD model uses the truncated pseudo-inverse formula

$$\tilde{\mathbf{A}} = \mathbf{Y} \mathbf{V}_r \Sigma_r^{-1} \mathbf{U}_r^T$$

to reconstruct "exactly" all non-zero modes Λ and Φ and thus perfectly reconstruct

$$= \Phi \Lambda \Phi^+.$$

All students contributed equally to this work. Names are listed in order of descending soccer fanaticism.

The Optimized DMD model, also called exponential fitting, uses the matrix

$$\mathbf{X} = [x_1 \quad \dots \quad x_m] \in \mathbb{R}^{n \times m}, x_i = x(t_i)$$

to solve

$$\min_{\substack{B \in \mathbb{C}^r \\ \alpha \in \mathbb{C}^{n \times r}}} \|\mathbf{X} - \mathbf{B}\theta(\alpha)\|_F$$

where

$$\theta(\alpha) = \begin{bmatrix} e^{\alpha_1 t_1} & \dots & e^{\alpha_1 t_n} \\ \vdots & & \vdots \\ e^{\alpha_r t_1} & \dots & e^{\alpha_r t_n} \end{bmatrix}.$$

This method of variable projection allows us to solve perfectly for

$$\mathbf{B} = \mathbf{X}\theta(\alpha)^+$$

and thus substitute

$$\min_{\alpha \in \mathbb{C}^r} \|\mathbf{X} - \mathbf{X}\theta(\alpha)^+\theta(\alpha)\|_F.$$

Now, the Extended DMD [2], or EDMD, is used for non-linear observables ψ . Assuming that

$$\psi : \mathbb{R}^n \rightarrow \mathbb{R}^d$$

is fixed, we observe $m + 1$ snapshots of

$$x_1, \dots, x_{m+1} \in \mathbb{R}^n$$

and define matrices

$$\mathbf{X} = \begin{bmatrix} \left| \psi(x_1) \right. & \dots & \left| \psi(x_m) \right. \\ \left| \right. & & \left| \right. \end{bmatrix} \in \mathbb{R}^{d \times m}$$

and

$$\mathbf{Y} = \begin{bmatrix} \left| \psi(x_2) \right. & \dots & \left| \psi(x_{m+1}) \right. \\ \left| \right. & & \left| \right. \end{bmatrix} \in \mathbb{R}^{d \times m}.$$

The EDMD model now learns

$$\mathbf{A} \in \operatorname{argmin}_{\mathbf{A} \in \mathbb{R}^{d \times d}} \|\mathbf{Y} - \mathbf{A}\mathbf{X}\|_F$$

$$\mathbf{A} = \mathbf{Y}\mathbf{X}^+.$$

Recall, $\mathbf{X} = \mathbf{U}\Sigma\mathbf{V}^T$ and $\mathbf{X}^+ = \mathbf{V}\Sigma^{-1}\mathbf{U}^T$ and notice that this is equivalent to classical DMD when $\psi = \mathbf{I}$. EDMD takes rank- r singular value decomposition $\mathbf{X} \approx \mathbf{U}_r \Sigma_r \mathbf{V}_r^T$ and computes $\mathbf{A} = \mathbf{Y}\mathbf{V}_r \Sigma_r^{-1} \mathbf{U}_r^T$ with complexity $\mathcal{O}(dmr)$. Note that it does not form this matrix but only computes its modes. There are two cases of interest:

	<u>coordinates</u>		<u>time</u>
Extended DMD (EDMD)	d	$\ll$	m
Kernel DMD (KDMD)	d	$\gg$	m

Note that the DMD model scales linearly in d and m , and that EDMD, KDMD are particularly useful when d or m are very large, respectively.

We start with $d \gg m$ and $\mathbf{X}_{k+1} = \mathbf{A}^T \mathbf{X}_k$. First, we take a look at the eigenvalues of

$$\mathbf{A} = \mathbf{X}^{+T} \mathbf{Y}^T.$$

Taking rank- r SVD

$$\mathbf{X} = \mathbf{U}_r \Sigma_r \mathbf{V}_r^T,$$

we have

$$\mathbf{X}^{+T} = (\mathbf{V}_r \Sigma_r^{-1} \mathbf{U}_r^T)^T = \mathbf{U}_r \Sigma_r^{-1} \mathbf{V}_r^T.$$

Thus, we have

$$\mathbf{A} = \mathbf{U}_r \Sigma_r^{-1} \mathbf{V}_r^T \mathbf{Y}.$$

Here, note that this is our usual DMD estimate transposed and that the range of $\mathbf{A}$ is spanned by $\mathbf{U}_r$. For the spectrum of $\mathbf{A}$, note that

$$\mathbf{A}\varphi = \lambda\varphi \in \text{span}(\mathbf{U}_r)$$

and so $\forall \lambda \neq 0$, we have

$$\lambda \in \text{span}(\mathbf{U}_r)$$

and we can write

$$\varphi = \mathbf{U}_r \tilde{\varphi}$$

and then

$$\begin{aligned} \lambda \mathbf{U}_r \tilde{\varphi} &= \mathbf{A} \mathbf{U}_r \tilde{\varphi} \\ &= \mathbf{X}^{+T} \mathbf{Y}^T \mathbf{U}_r \tilde{\varphi} \\ &= \mathbf{U}_r \Sigma_r^{-1} \mathbf{V}_r^T \mathbf{Y}^T \mathbf{U}_R \tilde{\varphi} \\ &= \mathbf{U}_r (\Sigma_r^{-1} \mathbf{V}_r^T) (\mathbf{Y}^T \mathbf{U}_r \Sigma_r \mathbf{V}_r^T) (\mathbf{V}_r \Sigma_r^{-1} \tilde{\varphi}) \\ &= \mathbf{U}_r (\Sigma_r^{-1} \mathbf{V}_r^T) (\mathbf{Y}^T \mathbf{X}) (\mathbf{V}_r \Sigma_r^{-1}) \tilde{\varphi} \end{aligned}$$

And thus we have $\lambda \tilde{\varphi} = (\Sigma_r^{-1} \mathbf{V}_r^T) (\mathbf{Y}^T \mathbf{X}) (\mathbf{V}_r \Sigma_r^{-1})^T \tilde{\varphi}$, where $\mathbf{Y}^T \mathbf{X} \in \mathbb{R}^{n \times n}$.

2.2 Sensitivity Methods for ODEs

We have two techniques for learning ODE methods from data:

Optimize-then-Discretize, where we treat the model as continuous in time. We derive sensitivity equations for the loss function $L(\theta)$ by differentiating under the integral sign, yielding either forward sensitivity or adjoint/backward sensitivity. The forward sensitivity augments the state ODE with parameter-derivative equations and the adjoint/backward sensitivity solve an auxiliary adjoint ODE backward in time to compute $\nabla_\theta L$ in one go. Adjoint methods incur constant memory cost (via checkpointing) and remain efficient even when θ is high-dimensional.

Discretize-then-Optimize applies a fixed-step integrator to produce a discrete map, $z_{n+1} = \Phi_h(z_n; \theta)$. Gradients $\nabla_\theta L$ are then obtained by backpropagating through each discrete step, either via explicit discrete-adjoint formulas or automatic differentiation. Although this is usually easier to implement, this can have bias if the step size h is large or variable.

Both techniques admit forward- and backward-sensitivities:

Forward sensitivity carries $\frac{\partial z}{\partial \theta}$ alongside z but scales poorly with the number of parameters.

Backward/adjoint sensitivity solves for $\lambda(t)$ where

$$\begin{aligned} \dot{\lambda} &= -\left(\frac{\partial f}{\partial z}\right)^T \lambda \\ \lambda(T) &= \frac{\partial l}{\partial z}(z(T)) \end{aligned}$$

and then recovers $\nabla_\theta L$ by integrating $\frac{\partial f}{\partial \theta}$ against $\lambda(t)$.

For our project, we use the optimize-then-discrete technique, with continuous-adjoint sensitivity. This allows us to integrate the learned vector field with adaptive solvers, compute exact continuous-time gradients via a backward pass, and avoid the large memory footprints that discrete checkpointing can incur, which is important when fitting moderately large neural-parameter ODE models.

3 Numerical Experiments

To better understand the relative performance of EDMD and neural ODEs in modeling and predicting the dynamics of linearly immersible systems, we choose the classic Lotka-Volterra predator-prey model as a testbed. After examining the modeling capability of both models in a linearly immersible system, we explore how well such methods generalize to non-linearly immersible systems, specifically the Lorenz system. By studying these models under both of these environments, we believe that we can accurately compare the trade-offs across both methods.

3.1 Scoring Quality of Dynamics Modeling

In order to properly compare the performance of different modeling approaches, we require a measure of the ability of some modeling technique to capture the dynamics of our systems.

Towards this, we compute Root Mean Square Error (RMSE) between the ground-truth system trajectory, denoted by $x^{(i)}[t]$, and those reconstructed using some modeling technique, denoted by $\hat{x}^{(i)}[t]$, over $T = 200$ time steps as a function of time t . To identify the ground-truth system trajectory, we forward step the ground-truth system using a high-order explicit Runge-Kutta solver. Further discussion of this process is deferred to Appendix A.

We note that tracking prediction error across the entire simulation will help elucidate any differences over which different techniques excel or suffer.

Further, to get a measure across different initial conditions, we analyze $M = 10$ trajectories, each with independent initial conditions sampled from a uniform distribution over the parameter space. We describe the specific parameter spaces during our discussion of each systems explored in this study. The same M ground-truth trajectories are used for all of the modeling techniques, allowing for better comparison. Equation 1 presents the aforementioned scoring metric:

$$\text{RMSE}[t] = \sqrt{\frac{1}{M} \sum_{i=1}^M \|\hat{x}^{(i)}[t] - x^{(i)}[t]\|_2^2} \quad \text{for } t \in [T] \quad (1)$$

3.2 Lotka–Volterra Predator–Prey Model

Often one of the first non-linear dynamical systems introduced in a typical introductory differential equations course, the Lotka–Volterra equations describe the dynamics of coupled species populations, where one species (the predator) feeds off of the other species (the prey). We denote the predator and prey populations as y and x , respectively. We assume that the prey have unlimited food, water, and habitat, and so that, absent the predatory risk, the prey population may grow exponentially. The populations evolve over time according to Equation 2 and Equation 3.

$$\frac{dx}{dt} = \alpha x - \beta xy \quad (2)$$

$$\frac{dy}{dt} = \delta xy - \gamma y \quad (3)$$

The rate of exponential prey growth is parameterized by α , representing how the prey population would increase in the absence of predators. Predation occurs at a rate proportional to the product of the prey and predator populations, xy , reflecting how interactions between the two species become more likely as either population grows. The predator population also grows at a rate proportional to xy , but scaled by a distinct parameter δ , which captures how effectively encounters with prey translate into predator reproduction. Finally, the term $-\gamma y$ models the death rate of predators independent of prey availability.

For the purposes of our study, we use ground-truth parameters $\alpha = 1.1, \beta = 0.4, \delta = 0.1, \gamma = 0.4$ and randomly sample initial condition (population levels) from the uniform distribution over $[1, 6] \times [1, 6] \subset \mathbb{R}^2$. We forward evolve each trajectory over a time horizon of $T = 25.0$ using a time step of $\Delta t = 0.025$.

3.2.1 Modeling Lotka–Volterra Dynamics Via Kernelized Extended DMD

As previously discussed, we are interested in understanding how well we can model and predict the dynamics of a linearly immersible system, such as the Lotka–Volterra predator–prey model, using extended DMD (EDMD), specifically kernelized EDMD [3, 4]. This variation of EDMD enables us to take advantage of the enhanced capability of EDMD to capture non-linear dynamics in the state space without having to worry about constructing the appropriate basis or dealing with explicitly lifting, making use of the classic kernel “trick” from machine learning theory.

However, there is still some room for exploration in selecting the appropriate kernel function. In configuring our experiments, we primarily played around with the Radial Basis Function (‘rbf’) kernel and the Polynomial (‘poly’) kernel (used in the original kernelized EDMD paper [3]) for

various polynomial degrees. We noticed that general performance under the Radial Basis Function kernel exceeded that under the Polynomial kernel, even for various polynomial degrees. Such result makes intuitive sense, as the RBF kernel is notably more expressive: it is commonly understood to implicitly project into an infinite-dimensional space. Based on these observations and intuition, we elected to use the RBF kernel for our experiments.

With the help of the kernel trick, the computational cost of EDMD is directly comparable to that of classic DMD (note that this is equivalent to endowing kernelized EDMD with the ‘linear’ kernel), and so we also compare performance with that of the classic DMD approach as well (see Figure 3).

In addition to selecting an appropriate kernel, we also controlled the effective dimensionality of the Koopman operator approximation by specifying the `svd_rank` parameter. We used a fixed rank of 10 for all experiments, which we found to be somewhere between stability and expressiveness. Admittedly, we did not play around with this parameter too much. Attempts to eliminate truncation resulted in numerical instability or divergence in the reconstruction, likely due to the inclusion of near-zero singular values.

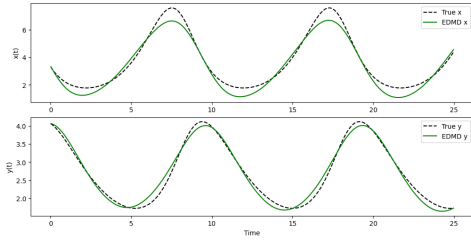


Figure 1: LV Reconstructed Via EDMD

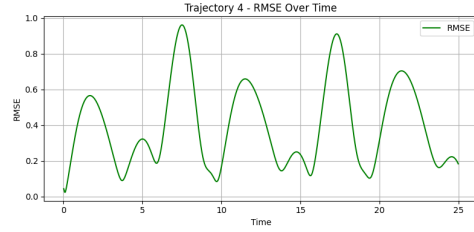


Figure 2: Single Trajectory RMSE

In Figure 1 and Figure 2, we present a representative Lotka–Volterra trajectory reconstructed using the kernelized EDMD method described above. In particular, the reconstruction closely follows the dynamics of the population based on ground truth, in contrast to the poorer fit produced by standard DMD (shown in Figure 3). This is further supported by the relatively low RMSE for the trajectory. These results suggest that EDMD is effective at capturing the underlying dynamics of the Lotka–Volterra system. Figure 3 shows a trajectory reconstructed using standard DMD for the same

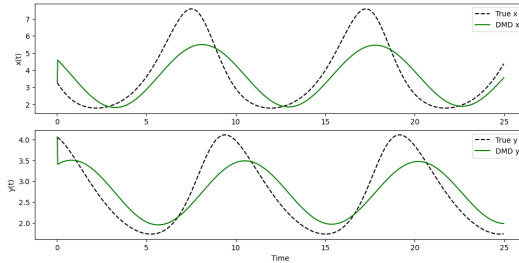


Figure 3: LV Reconstructed Via DMD

system considered previously. Notably, the reconstruction fits the ground-truth trajectory poorly and fails to accurately capture the initial conditions. While this example is shown as a single reference, it is representative of a broader trend observed across all trajectories we analyzed. These results highlight the limitations of standard DMD and underscore the need for using EDMD to effectively capture the dynamics of the Lotka–Volterra system as expected.

3.2.2 Modeling Lotka–Volterra Dynamics Via Neural ODEs

Recent advancements in deep learning have enabled the modeling of complex dynamical systems by directly learning differential equations from data. We dip our toes into this exciting direction by using rudimentary neural ODEs to model the systems considered in our study.

To model the system dynamics, we construct a neural network to learn a function $f_\theta(z) \approx \frac{dz}{dt}$, where the quantity $z \in \mathbb{R}^2$ represents the population levels, by taking the current populations z as input, and then outputting a predicted time derivative.

Our neural networks consists of a single hidden layer with 64 units and a hyperbolic tangent activation function, which allows the modeling of nonlinear dynamics while remaining as simple as possible. Further discussion of the model details is deferred to Appendix B. We use the same experimental setup and ground-truth trajectory data as in the EDMD experiments.

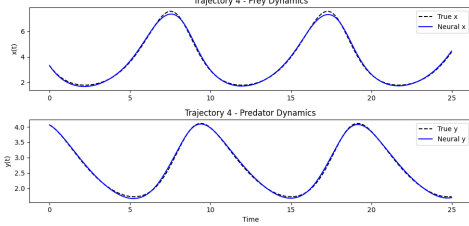


Figure 4: LV Reconstructed Via Neural ODE

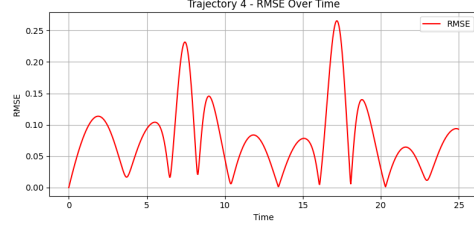


Figure 5: Single Trajectory RMSE

In Figure 4 and Figure 5, we show the dynamics of the system learned by the neural ODE from the same ground-truth trajectory as in Figure 1. Notice that the neural ODE demonstrates a remarkable ability to reconstruct the trajectory, as evidenced by the predicted trajectory that hugs the ground-truth and the quite low RMSE values. Of particular note is the fact that while the magnitude of the RMSE values differ between the EDMD reconstruction (Figure 2) and that seen here (Figure 5), there is a noticeable similarity between the shapes of the two RMSE plots, suggesting that certain portions of the trajectory may be more difficult to model than others.

Although the performance of the neural ODE trained on this trajectory exceeds the performance of the EDMD on the same trajectory, this was not the case on all trajectories.

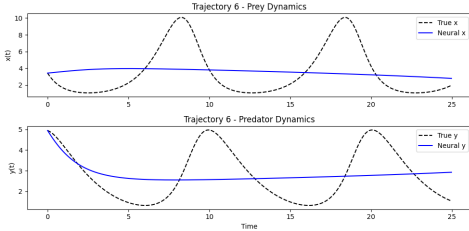


Figure 6: Hard Trajectory For Neural ODE

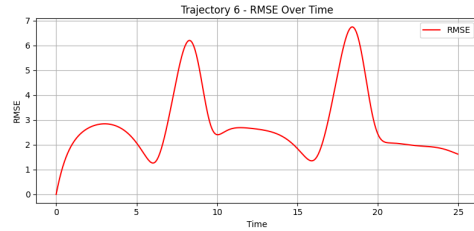


Figure 7: Significantly Worse RMSE

For some trajectories, the neural ODE struggled to capture any meaningful dynamics. One such trajectory is presented in Figure 6 and Figure 7. Notably absent in the reconstructed trajectories are the characteristic oscillations typically observed in the population levels. Instead, the model produces a trajectory that cuts directly through the oscillatory patterns, indicating a failure to learn the underlying system behavior and instead just selecting a path that averages out the oscillations, potentially corresponding to a local loss minimum. Investigating alternative optimization methods for this learning problem may be worth exploring in the future.

3.2.3 Understanding Differences In Modeling Lotka-Volterra Dynamics

Having modeled identical trajectories for the Lotka-Volterra system using both EDMD and a neural ODE, a natural question is how the two approaches compare as far as reconstruction quality. In Figure 8, we present the average RMSE across all Lotka-Volterra trajectories for both methods. On average, the RMSE values are comparable between the neural ODE and EDMD approaches. However, it is important to interpret this result in light of our earlier observations: the neural ODE seemed to either nail the system dynamics or miss the mark entirely, suggesting greater variability in reconstruction quality.

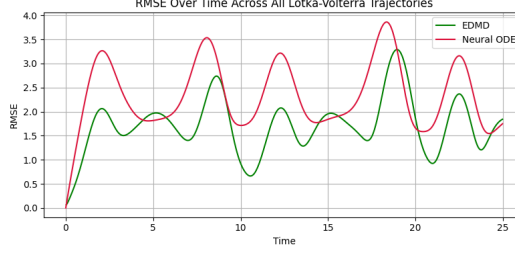


Figure 8: Comparing LV Average RMSE Across Methods

3.3 Lorenz System

The Lorenz system is a three-dimensional system of differential equations that describes a simple atmospheric model. The system is popular in chaos theory because it is sensitive to initial conditions, where small perturbations cause substantially different outcomes (see *the butterfly effect*). The equations governing the Lorenz system are:

$$\begin{aligned}\frac{dx}{dt} &= \sigma(y - x) \\ \frac{dy}{dt} &= x(\rho - z) - y \\ \frac{dz}{dt} &= xy - \beta z\end{aligned}\tag{4}$$

Here, x, y, z represent the convective flow, horizontal temperature difference, and vertical temperature deviation, respectively. The constants σ, ρ, β are the Prandtl number, Rayleigh number, and geometric factor. The first equation shows that when the temperature difference y exceeds the current flow x , the fluid accelerates, and when $x > y$, the flow slows. The second equation describes how y grows when convective motion x amplifies the vertical imbalance $(\rho - z)$, and it decays over time. Lastly, the third equation captures how z has higher deviation when there is both motion and temperature difference. The “minus” factors in the last two equations are to account for dissipation [5].

Importantly, when it was first posed, the system was studied under parameter values $\sigma = 10, \beta = \frac{8}{3}$, and $\rho = 28$, which leads to the butterfly attractor [6]. We examine the system under such classical parameter values. As in the Lotka-Volterra system, we randomly sample the initial condition (starting point), this time from $[-5, 5] \times [-5, 5] \times [-5, 5] \subset \mathbb{R}^3$. We forward evolve each trajectory over a time horizon of $T = 25.0$ using a time step of $\Delta t = 0.025$.

3.3.1 Modeling Lorenz System Dynamics Via Kernelized Extended DMD

To evaluate the ability of EDMD to model the dynamics of the Lorenz system, we used the same EDMD parameters as in the Lotka-Volterra experiments. However, we observed a noticeable degradation in reconstruction quality. In particular, one of the ten trajectories exhibited exceptionally poor reconstruction quality.

In reconstructing this particular trajectory, EDMD fits a path that spirals out of control away from the ground-truth trajectory (Figure 9), which understandably leads to extremely poor RMSE values (Figure 10). It should be noted that swapping out the Radial Basis Function kernel for a Polynomial kernel of degree 2 eliminates this divergence in trajectory. This suggests that using a highly expressive kernel in the context of a chaotic system may introduce instability, even just during trajectory reconstruction. However, not all trajectories that we ran EDMD on demonstrated such behavior.

The particular trajectory shown above in Figure 11 and Figure 12 is representative of the typical trajectory reconstructed by EDMD. Notice how the reconstructed trajectory does not quite follow the ground-truth, it still moves toward the general region of the attractor, capturing some qualitative features of the system’s global structure. However, in doing so, the reconstructed trajectory seems to meander and lack much detail, potentially reflecting the challenges for EDMD to learn the complex dynamics of the Lorenz system.

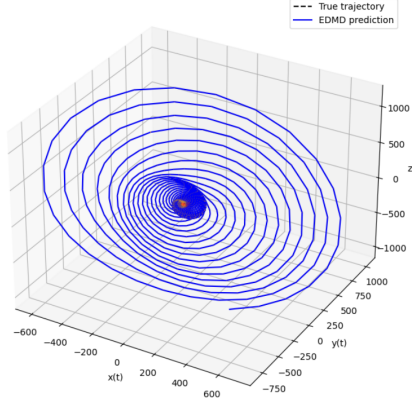


Figure 9: Hard Lorenz Trajectory For EDMD

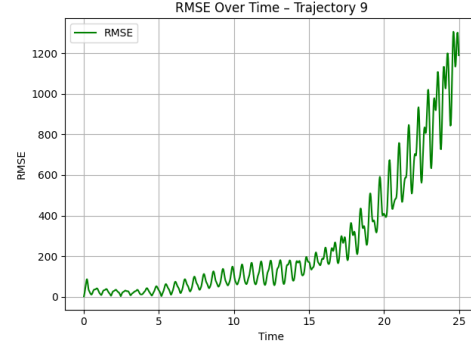


Figure 10: Exceptionally High RMSE Values

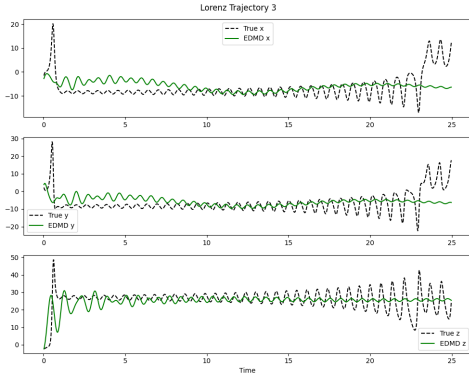


Figure 11: EDMD Lorenz Trajectory

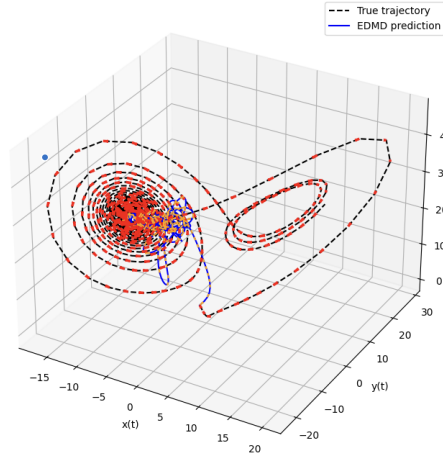


Figure 12: Quiver Plot of Trajectory

3.3.2 Modeling Lorenz System Dynamics Via Neural ODEs

Similar to our exploration of modeling the dynamics of the Lorenz system using EDMD, we apply the same basic neural ODEs as for the Lotka-Volterra system. While neural ODEs did not manage to capture the dynamics of certain trajectories as well as it did for a select few Lotka-Volterra trajectories, some patterns remerge.

Figure 13 and Figure 14 show a representative trajectory of the Lorenz system as reconstructed using neural ODEs. Notice how the reconstructed path appears to head almost directly from the initial condition to the “eye” of the attractor, bypassing the characteristic oscillations, an effect similar to the oscillation-splitting behavior we observed earlier in Figure 6. As before, this kind of shortcut behavior likely corresponds to a local minimum found by the optimizer, where the model sacrifices fine-grained accuracy for a broadly correct location in phase space. Given the strong region of attraction in the Lorenz system, it may be worth considering a loss function that more heavily penalizes early trajectory error, to discourage the model from simply making a beeline to where the system settles, and thus expending less effort matching the true short-term dynamics.

3.3.3 Understanding Differences In Modeling Lorenz System Dynamics

Once again, we aim to compare how the two modeling approaches, EDMD and neural ODEs, perform in reconstructing the trajectories of the Lorenz system. Given that one of the EDMD reconstructions resulted in an exceptionally high RMSE relative to all other trajectories, we include

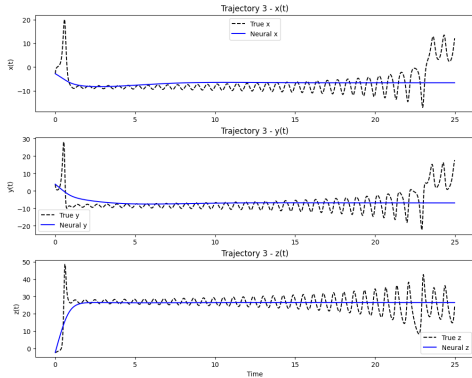


Figure 13: Neural ODE Lorenz Trajectory

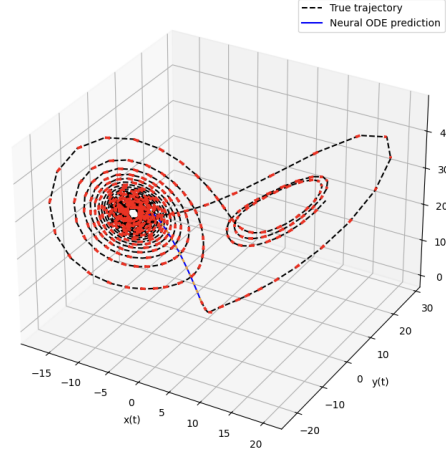


Figure 14: Quiver Plot of Trajectory

two comparisons: the average RMSE across all Lorenz trajectories for both methods (Figure 15) and a second comparison that excludes this outlier trajectory (Figure 16).

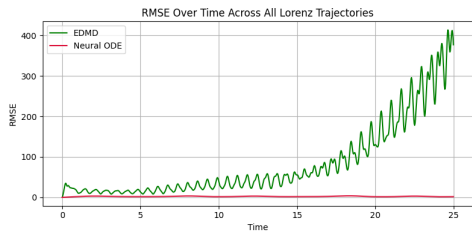


Figure 15: Comparing Lorenz RMSEs

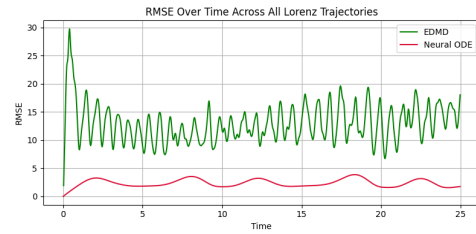


Figure 16: Lorenz RMSEs Without Trajectory From Figure 9

Clearly, the RMSE of the outlier trajectory dominates the left figure. However, in the right figure, we see that the RMSEs are closer in magnitude, but still the neural ODEs appear to outperform EDMD. This is likely due to the ability of the neural ODEs to avoid the meandering paths EDMD would reconstruct.

From this, a common pattern emerges: both EDMD and neural ODEs struggle to capture the short-term dynamics of the Lorenz system, instead offering only rough approximations of the system's long-term behavior. Further, neither model is particularly expressive, which may limit their ability to capture the complex dynamics of chaotic systems, such as Lorenz. Thus, it may be fruitful to explore more expressive or, perhaps, hybrid modeling approaches.

4 Discussions and Conclusion

From our experiments conducted, we find distinct strengths and weaknesses in each modeling approach. For the Lotka–Volterra system (first system studied), both EDMD and neural ODEs manage to reconstruct the dynamics with reasonable accuracy. Kernelized EDMD, especially with the RBF kernel, appeared to capture the oscillatory behavior reliably across the various initial conditions. Its performance remains stable without requiring extensive hyperparameter tuning. Neural ODEs, while capable of producing very accurate reconstructions in some cases, show noticeable variability across trials. In several trajectories, the model fails to learn the cyclic structure and instead generates paths that smooth out the natural oscillations, indicating sensitivity to training dynamics and initialization as seen in Figure 6 and Figure 7.

In the case of the Lorenz system, both methods encounter significant challenges. EDMD often produced meandering or diverging trajectories as the expressiveness of the kernel sometimes works

against the model, leading to instability during reconstruction. Neural ODEs frequently take a more direct path to the central region of the attractor, bypassing much of the transient behavior. While this may minimize the loss, it results in poor short-term behavior and fails to capture the system’s sensitivity to initial conditions. Neither model succeeds in fully capturing the complex structure of the Lorenz dynamics, and both suffer from limitations in expressiveness and stability.

Overall, while our results do show that EDMD was generally stable and performed well on the Lotka–Volterra system, it broke down on the Lorenz system, with some trajectories diverging completely. Neural ODEs showed stronger potential to capture complex dynamics but lacked consistency, sometimes missing key structural features or collapsing into oversimplified paths. These contrasting failure modes suggest that neither approach is sufficient on its own, and prove demand for new methods that can learn rich, nonlinear behavior without sacrificing stability across a range of systems.

References

- [1] George Stepaniants. Data-driven methods for dynamical systems. Lecture notes, 2025. Course notes for ACM 270, California Institute of Technology. Unpublished manuscript.
- [2] Matthew O. Williams, Ioannis G. Kevrekidis, and Clarence W. Rowley. A data-driven approximation of the koopman operator: Extending dynamic mode decomposition. *Journal of Nonlinear Science*, 25(6):1307–1346, June 2015. ISSN 1432-1467. doi: 10.1007/s00332-015-9258-5. URL <http://dx.doi.org/10.1007/s00332-015-9258-5>.
- [3] Matthew O. Williams, Clarence W. Rowley, and Ioannis G. Kevrekidis. A kernel-based approach to data-driven koopman spectral analysis, 2015. URL <https://arxiv.org/abs/1411.2260>.
- [4] Sara M. Ichinaga, Francesco Andreuzzi, Nicola Demo, Marco Tezzele, Karl Lapo, Gianluigi Rozza, Steven L. Brunton, and J. Nathan Kutz. Pydmd: A python package for robust dynamic mode decomposition, 2024. URL <https://arxiv.org/abs/2402.07463>.
- [5] Mark McClure. The lorenz experiment. <https://marksmath.org/visualization/LorenzExperiment/>, n.d. Accessed: 2025-05-27.
- [6] Jeremy P Parker. The lorenz system as a gradient-like system. *Nonlinearity*, 37(9):095022, 2024.
- [7] Pauli Virtanen, Ralf Gommers, Travis E Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J van der Walt, Matthew Brett, Joshua Wilson, K Jarrod Millman, Nikolay Mayorov, Andrew R J Nelson, Eric Jones, Robert Kern, Eric Larson, CJ Carey, Ilhan Polat, Yu Feng, Eric W Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E A Quintero, Charles R Harris, Anne M Archibald, Antônio H Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. Scipy 1.0: fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.
- [8] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library, 2019. URL <https://arxiv.org/abs/1912.01703>.
- [9] Ricky T. Q. Chen. torchdiffeq, 2018. URL <https://github.com/rtqichen/torchdiffeq>.

A Solving For Ground-Truth System Trajectories

A crucial component of the numerical analysis conducted in this project are the ground-truth system trajectories, which we use to both learn the dynamics of the two systems as well as compare the reconstructed trajectories to. Given that we select the true parameters values for both systems, we can construct the ground-truth system trajectories by simply forward stepping the system for the designated period of time, and then store those values as our data to then learn to reconstruct.

To perform this integration, we leverage the numerical ODE solvers implemented by the `scipy` package [7]. More specifically, we use the DOP853 method via the `solve_ivp` function interface. DOP853 is a high-order (8th-order) explicit Runge-Kutta method.

We used the default relative tolerance `rtol` of 10^{-3} and the absolute tolerance `atol` of 10^{-6} . The integration is performed over the full duration of the trajectory window with sampling at fixed time intervals. This yields the time-series data for each system that we treat as ground truth.

B Additional Neural ODE Details

The model for the neural ODEs is implemented using the PyTorch [8] and `torchdiffeq` [9] libraries. The neural ODE is trained to minimize the mean squared error between the predicted and ground-truth trajectory using the Adam optimizer over 2,000 epochs. The number of epochs was selected after a bit of experimenting. Towards numerical stability during learning, we normalize the trajectories prior to training.

All training was completed on an M1 Pro with 16GB RAM. Training each model on a trajectory took approximately 30 minutes, and so all models were left to train sequentially overnight.

C Declared Usage of Large Language Models (LLMs)

We leveraged LLMs to assist in coding numerical experiments as allowed per the course syllabus.